Technical System Documentation

ChainCert

Blockchain-Based Credential Issuance and Verification Platform

March 25, 2026

CAP-2459-IT

Table of Contents

1. Introduction

2. System Overview

3. System Architecture

4. Blockchain Network Architecture

5. Infrastructure Deployment Model

6. System Data Flow

7. Governance Model

8. Backend Architecture

9. API Architecture

10. Smart Contract (Chaincode) Design

11. Credential Lifecycle

12. Database Design

13. Security Model

14. System Directory Structure

15. System Limitations

16. Access Control Model

17. On-Chain vs Off-Chain Data Design

18. Key Transaction Flows

19. Scalability Considerations

20. Testing Strategy

References

Appendix A — Chaincode CLI Command Reference

Appendix B — Backend File Structure

Appendix C — Chaincode Redeployment Instructions

1. Introduction

This document describes the technical architecture, system components, backend implementation, blockchain integration, database design, and API structure of the ChainCert platform — a blockchain-based credential issuance and verification system developed as part of the CAP-2459 IT Capstone Project.

This document is intended for:

- System administrators responsible for deploying the platform
- Developers maintaining or extending the system
- Technical evaluators reviewing the architecture of the solution

The documentation details how the ChainCert platform integrates React frontend applications, Node.js backend services, Hyperledger Fabric blockchain infrastructure, and CouchDB off-chain databases to implement a secure and tamper-resistant credential ecosystem.

The goal of the system is to ensure that digital credentials issued by accredited organizations are verifiable, immutable, and resistant to tampering or forgery.

2. System Overview

ChainCert is a blockchain-based credential issuance and verification platform designed to ensure the authenticity and integrity of digital credentials.

Traditional credential systems rely on centralized databases that can be modified, tampered with, or forged. This creates verification challenges for employers, educational institutions, and organizations attempting to validate credentials.

The ChainCert platform addresses these issues by combining:

- A web-based credential management platform
- A permissioned blockchain network powered by Hyperledger Fabric
- Off-chain databases for application data

Credentials issued through the system are recorded as transactions on a distributed ledger, ensuring immutability, traceability, and auditability.

Platform Capabilities

Issuers

- Create credential templates
- Issue credentials to holders
- Revoke credentials when necessary

Credential Holders

- Manage credentials through a digital wallet
- Control credential visibility (public/private)
- Share profile via shareable links

Verifiers

- Verify credentials using credential IDs or public verification interfaces

Administrators

- Govern issuer onboarding and platform operations
- Monitor system-wide credential activity
- Run CouchDB vs blockchain integrity checks

3. System Architecture

The ChainCert platform follows a multi-layered architecture composed of four primary layers.

Frontend Layer

The frontend layer is implemented using React.js and provides the user interface for all system roles, including administrators, issuers, credential holders, and verifiers. The frontend communicates with the backend through RESTful API endpoints.

Backend API Layer

The backend system is implemented using Node.js with the Express framework. Responsibilities include: authentication and authorization, issuer onboarding and governance, credential issuance management, wallet management, verification services, blockchain transaction orchestration, admin notifications, profile sharing, and CouchDB vs blockchain integrity checking.

The backend interacts with both Hyperledger Fabric blockchain and CouchDB off-chain storage.

Blockchain Layer

The blockchain layer uses Hyperledger Fabric, a permissioned blockchain framework designed for enterprise applications. Blockchain responsibilities include recording credential issuance, revocation, expiration, verification logs, and issuer governance decisions. Smart contracts (chaincode) enforce the credential lifecycle logic.

Database Layer

The system uses CouchDB as an off-chain database layer for storing application data, including user accounts, issuer registrations, notifications (user and admin), credential wallet caches, and profile sharing tokens. The blockchain ledger stores only credential integrity records and governance logs, while operational data remains off-chain.

Containerized Deployment

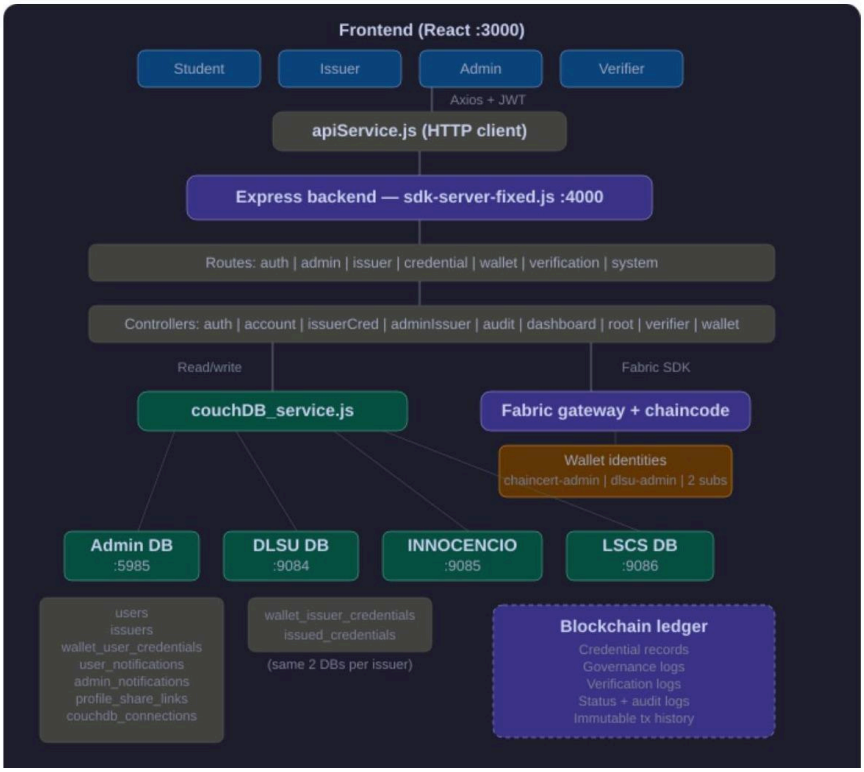
The core infrastructure components of ChainCert are deployed using Docker containers. This includes Hyperledger Fabric peer nodes, ordering services, Fabric Certificate Authorities, and CouchDB

instances. Containerization provides isolated execution environments, simplifies setup and replication, and ensures consistent deployment across development and testing environments.

3.1 System Architecture Diagram

The following diagram illustrates the complete application architecture, showing how the frontend, backend, CouchDB instances, and blockchain layer connect.

Figure 1: Full application architecture — React frontend, Express backend, CouchDB instances, and Hyperledger Fabric blockchain



4. Blockchain Network Architecture

ChainCert uses Hyperledger Fabric as the blockchain infrastructure. Key network components include:

Peer Nodes

Peer nodes have two jobs: (1) execute chaincode when a transaction proposal arrives and return a signed endorsement, and (2) maintain their own copy of the ledger and world state. Each peer's world state database (CouchDB) is a snapshot of the current value of every key, so the system does not need to replay the entire chain to determine the current status of a credential. Each participating organization operates a peer node within the network.

Ordering Service

The ordering service receives endorsed transactions from the backend, sequences them into blocks, and distributes those blocks to all peers on the channel. The orderer does not execute or validate transaction logic — it only determines the order in which transactions are committed. This ensures all peers process the same transactions in the same sequence, preventing ledger divergence.

Fabric Certificate Authorities

Fabric CA servers issue X.509 certificates used for identity authentication, network access control, and transaction signing. Each organization runs its own CA. The CA handles two operations: Register (creates an entry in the CA database with a name and secret) and Enroll (the identity proves it knows the secret, and the CA issues a signed certificate and private key). The orderer has its own CA because it is its own organizational entity — it needs a TLS certificate to authenticate itself to peers, sign blocks, and establish encrypted connections.

Channels

Channels provide private communication layers where transactions are executed and recorded. The current ChainCert deployment uses a channel named chaincert-channel. All three peer nodes are joined to this channel.

Membership Service Providers (MSP)

MSPs define organization identities, network participants, and permission boundaries. The current deployment includes: Org1MSP, Org2MSP, DLSUMSP, and OrdererMSP.

4.1 Containerized Network Components

The Hyperledger Fabric network in ChainCert is deployed using Docker containers. Each major blockchain component runs as an isolated containerized service.

Peer Containers

These containers host the blockchain peers for participating organizations and are responsible for maintaining ledger copies and executing chaincode.

- peer0.org1.example.com (port 7051)
- peer0.org2.example.com (port 9051)
- peer0.dlsu.example.com (port 12051)

Ordering Service Container

- orderer.example.com (port 7050)

Fabric CA Containers

- ca_org1 (port 7054)
- ca_org2 (port 8054)
- ca_dlsu (port 12054)
- ca_orderer (port 9054)

CouchDB Containers

- couchdb0 (Org1 peer state DB)
- couchdb1 (Org2 peer state DB)
- couchdb_dlsu (DLSU peer state DB)
- couchdb-admin (Admin application DB — port 5985)
- couchdb_dlsu_issuer (DLSU issuer DB — port 9084)
- couchdb_innocencio (INNOCENCIO issuer DB — port 9085)
- couchdb_lasallecomputersociety (La Salle Computer Society issuer DB — port 9086)

4.2 Blockchain Network Diagram

The following diagram shows all Fabric network components including peers, orderer, certificate authorities, peer state databases, and MSP boundaries.

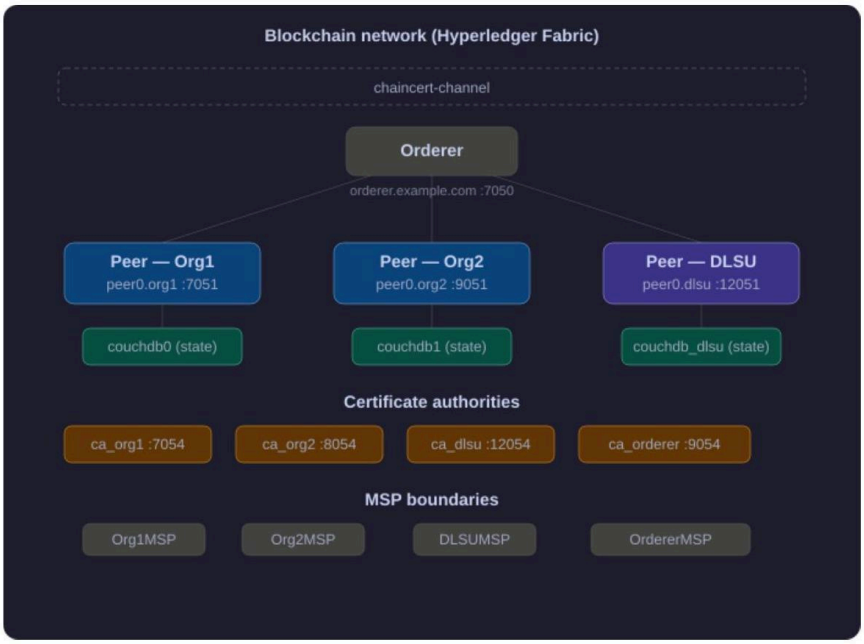


Figure 2: Blockchain network architecture — orderer, 3 peers, 4 CAs, 3 peer state DBs, and MSP boundaries

Figure 2: Blockchain network architecture — orderer, 3 peers, 4 CAs, 3 peer state DBs, and MSP boundaries

5. Infrastructure Deployment Model

The ChainCert platform is architecturally designed to support a consortium blockchain deployment model. In this model, multiple trusted organizations may participate in a shared Hyperledger Fabric network, and root authorities may operate their own peer nodes and supporting infrastructure.

However, in the current prototype implementation, the blockchain infrastructure is centrally deployed and managed by the ChainCert team. This includes the orchestration of Docker containers for peer nodes, ordering services, Fabric CA servers, and CouchDB instances. Under this prototype setup:

- ChainCert hosts the blockchain network environment
- ChainCert manages node deployment and service orchestration
- ChainCert performs organization onboarding and technical activation
- Logical separation of organizations is preserved through distinct MSPs, identities, peer containers, and issuer-specific databases

This centrally managed deployment was adopted to simplify development, reduce onboarding complexity, and maintain a controlled testing environment during the prototype phase.

The system remains compatible with a future consortium expansion model, where root authorities can deploy and manage their own infrastructure using a provided deployment kit.

5.1 Platform Model vs Consortium Model

The following diagram contrasts the current platform-managed model (where ChainCert controls all infrastructure and signs channel updates alone) with the future consortium model (where each organization deploys its own infrastructure and channel updates require multi-org cryptographic signatures).



Figure 3: Current platform model vs future consortium model — governance and infrastructure comparison

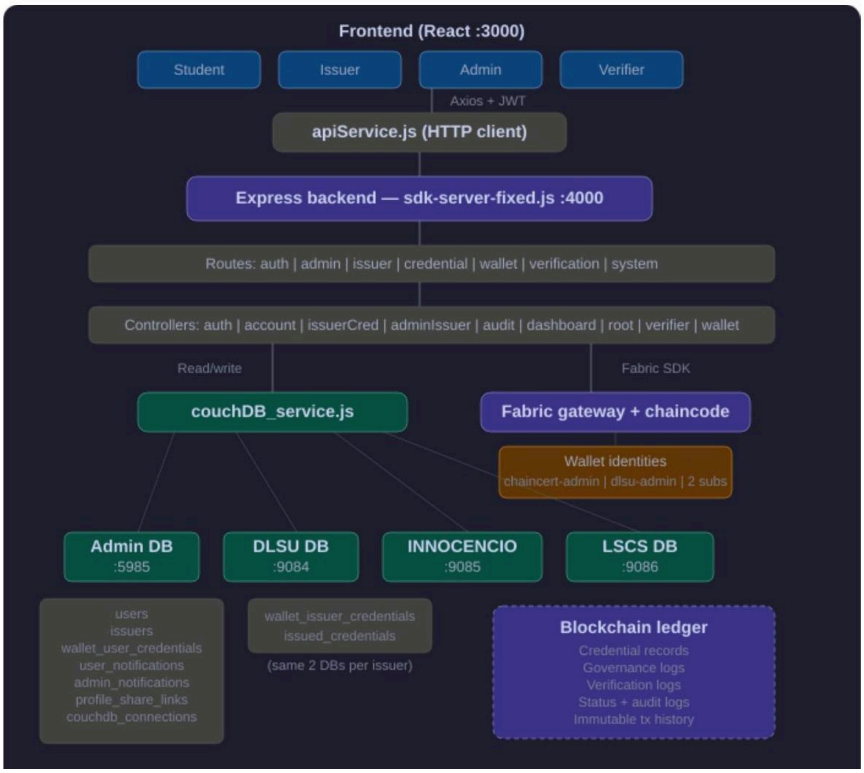
6. System Data Flow

A typical request within the system follows the sequence below.

Credential Issuance Flow

- The Issuer interacts with the React frontend dashboard
- The frontend sends an HTTP request to the backend API
- Express routes forward the request to the appropriate controller
- The controller performs validation and business logic
- CouchDB is queried for the required data
- The Hyperledger Fabric SDK submits a transaction to the blockchain
- The chaincode processes the transaction
- The transaction is committed to the ledger
- The backend updates off-chain wallet records
- A credential notification is created for the holder
- The response is returned to the frontend

6.1 System Data Flow Diagram



6.2 Fabric Transaction Lifecycle

When a blockchain transaction is submitted (e.g., credential issuance), it passes through a multi-step lifecycle within the Fabric network:

- **Step 1 — Proposal:** The backend builds a transaction proposal signed by the appropriate wallet identity (e.g., delasalleuniversity-admin for DLSU credential operations) and sends it to the endorsing peers.
- **Step 2 — Endorsement:** Each endorsing peer (e.g., Org1 + DLSU) independently executes the chaincode function and returns a signed endorsement. Both peers must agree for the endorsement policy to be satisfied.
- **Step 3 — Collection:** The backend collects endorsements from all required peers and packages them into a transaction envelope.
- **Step 4 — Ordering:** The orderer receives the endorsed transaction, sequences it into a block with other transactions, and delivers the block to all peers. The orderer does NOT re-execute the chaincode — it only orders and batches.

- **Step 5 — Validation and Commit:** Each peer independently validates the block (checks endorsement signatures, detects conflicts) and appends the block to its local ledger copy. The peer's world state CouchDB is updated to reflect the new credential status.
- **Step 6 — Result:** All three peers now hold identical ledger copies containing the same transaction. If any single peer's data is tampered with, it would differ from the other two — this is the tamper-evidence guarantee.

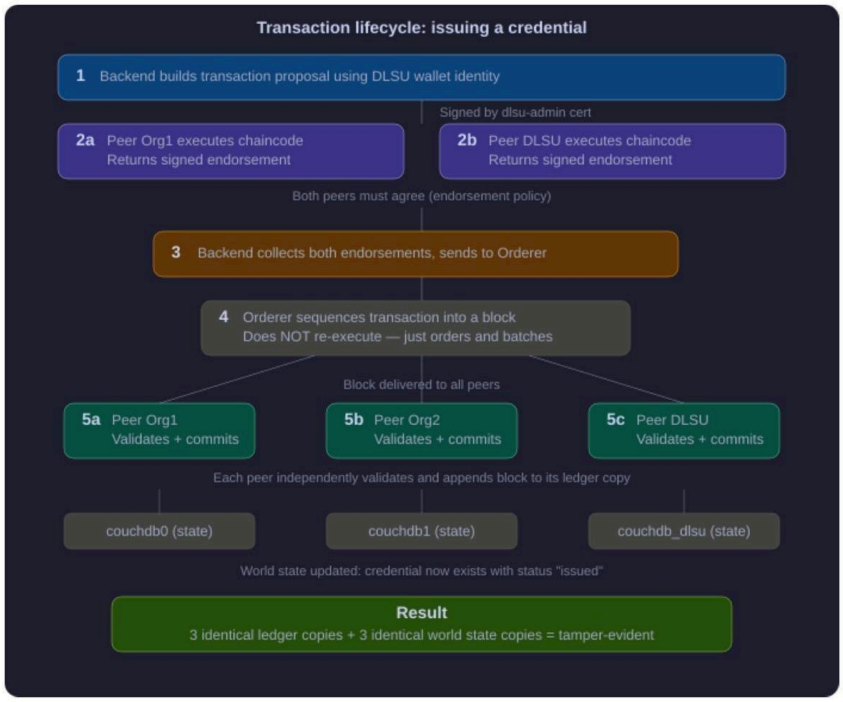


Figure 4: Transaction lifecycle for credential issuance through the Fabric network

7. Governance Model

ChainCert implements a hierarchical governance structure.

Administrator

Platform administrators oversee the entire system. Responsibilities include: approving issuer registrations, disabling or enabling issuers and users, monitoring credential issuance activity, running integrity checks, and managing admin notifications.

Root Authorities

Root authorities are trusted institutions that can create templates, issue credentials to holders, and may approve or reject subordinate issuers.

Subordinate Issuers

Subordinate issuers operate under root authorities and require a two-tier approval process before issuing credentials. First, the root authority must approve the subordinate application (setting rootApprovalStatus to approved). Only then can the platform administrator approve the issuer. The admin approval step is blocked until the root authority gate passes. This ensures that subordinate issuers are vetted by both their governing institution and the platform before gaining credential issuance privileges.

Credential Holders

Users who receive digital credentials. They manage credentials through the digital wallet system, control visibility, and share profiles via share links.

Verifiers

Entities such as employers or institutions that verify credential authenticity using the public verification interface.

8. Backend Architecture

The backend is structured using a controller-based architecture where each controller manages a specific system domain.

authController

Handles authentication operations including: user registration, user login, issuer login, and password changes. Authentication uses JWT tokens for API authorization.

accountController

Handles user account management including: retrieving user information, updating profiles (summary, visibility, username), profile picture uploads, generating credential profile sharing links (public and recipient-specific), OTP-based share link verification, share link revocation, forgot password (OTP via email), and password reset.

issuerOnboardingController

Manages issuer registration and onboarding. Responsibilities include: issuer registration submission (with file attachments stored as CouchDB attachments), issuer document validation, issuer document retrieval/download, and category-based document requirement validation.

issuerCredentialController

Handles credential management operations including: credential template creation (blockchain + CouchDB dual write), credential issuance (with holderName enrichment, templateId tracking, duplicate prevention, and holder notification), credential revocation (blockchain + CouchDB + holder notification), credential history retrieval (with holderName enrichment from wallet snapshots), eligible user filtering per template, and credential template listing.

adminIssuerController

Handles administrator governance actions related to issuers including: approving issuer applications (with Fabric identity assignment, CouchDB URL resolution, and blockchain logging), rejecting issuer applications (with blockchain logging), enabling and disabling issuers (with blockchain logging), and admin notifications for each decision.

adminAuditController

Handles administrative audit functions including: user account activation and deactivation (with blockchain logging), issuer status logs, user status logs, issuer decision logs (approval/rejection), root subordinate decision logs, and admin password change (with current password verification and audit logging).

adminDashboardController

Provides aggregated credential data for the administrator dashboard. Includes functionality for: retrieving system-wide credentials across all issuers (with type/issuer filters and holderName enrichment), admin notifications (get all, get unread, mark read, mark all read), automated credential expiration checks using scheduled cron jobs (midnight + manual trigger), and CouchDB vs blockchain integrity checking (field-by-field comparison using Get_CredentialTemplateOrIssuedCredential).

rootAuthorityController

Handles governance operations performed by root authorities including: reviewing subordinate issuer applications, approving subordinate issuers (with blockchain logging and admin notification), rejecting subordinate issuers (with blockchain logging and admin notification), retrieving subordinate issuer lists (all, pending, root-approved, root-rejected, approved, disabled), and retrieving subordinate decision logs from blockchain.

verifierController

Implements credential verification logic. Operations include: verifying credential validity (CouchDB + blockchain cross-check), retrieving verification statistics, recording verification events on the blockchain, and retrieving verification logs.

walletController

Manages credential holder wallet functionality including: retrieving wallet credentials, updating credential visibility (public/private), updating credential display order, updating issuer template visibility (templates only, not issued credentials), retrieving user notifications, retrieving unread user notifications, and marking notifications as read.

8.1 Fabric Identity Resolution

The backend resolves which Fabric wallet identity to use for blockchain transactions based on the acting user's role and issuer record. The resolution logic works as follows:

- **chaincert-admin** — Used for system-level operations: admin governance actions, credential verification, expiry processing, integrity checks, and any operation initiated by non-issuer roles.
- **delasalleuniversity-admin** — Used for DLSU root authority credential operations: template creation, credential issuance, and revocation when the acting issuer is DLSU.
- **delasalleuniversity-subordinate-innocencio** — Used for INNOCENCIO subordinate credential operations.
- **delasalleuniversity-subordinate-lasallecomputersociety** — Used for La Salle Computer Society subordinate credential operations.

The identity is resolved by looking up the approved issuer record in CouchDB (which contains fabricIdentity and orgMSP fields) and passing those to the getContractAndGateway helper function in sdk-server-fixed.js, which loads the corresponding wallet identity and establishes a Fabric gateway connection.

8.2 Issuer Document Requirements by Category

The system enforces category-specific document requirements during issuer registration. When an issuer selects their category, the frontend dynamically loads the required documents from the issuerRequirements.js configuration file. The admin reviews these documents before approving the issuer. There are five issuer categories, each with its own document set.

University / Higher Education Institution

#	Document	Description
1	**CHED / Government Recognition**	Copy of CHED Government or Institutional Recognition, and/or program recognition, or charter/ordinance for SUCs/LUCs, confirming authority to operate and grant degrees.
2	**Legal Personality Document**	SEC Certificate of Incorporation for private HEIs, or Republic Act/charter/ordinance establishing the HEI as a public institution.
3	**Board Resolution — ChainCert Authorization**	Governing board resolution approving participation in ChainCert and designating authorized officials to issue digital credentials.

4	**Quality Accreditation Certificate (Optional)**	Current accreditation certificate from a CHED-recognized accrediting body (e.g., PAASCU, PACUCOA, AACCUP) as a quality assurance signal.

Government Agency or Office

#	Document	Description
1	**Legal Creation Instrument**	Copy of the Republic Act, Executive Order, Administrative Order, or local ordinance formally creating the government agency or office.
2	**Mandate / Function Document**	Documented mandate (enabling law provisions, administrative code, or agency charter) showing authority to issue the relevant credentials.
3	**Office Order / Memorandum — ChainCert Authorization**	Office Order or Memorandum designating the specific unit and officials authorized to issue credentials via ChainCert.
4	**Sample Official Credential Template**	Specimen or template of the current government-issued certificate, license, or eligibility document that will be digitized.

Training Provider / Private Education Provider

#	Document	Description
1	**SEC or DTI Registration**	SEC Certificate of Incorporation for corporations/partnerships, or DTI Business Name Registration for sole proprietors.
2	**LGU Business / Mayor's Permit**	Latest business permit from the city/municipality confirming authority to operate at the specified address.
3	**TESDA CoPR / Program Approval**	TESDA Certificate of Program Registration (CoPR) for TVET programs, or equivalent written approval/accreditation for non-TESDA programs.
4	**Program Brief / Training Regulation Alignment**	Program description including learning outcomes, curriculum, assessment methods, and alignment with TESDA Training Regulations.

Industry Certifier / Professional Certification Body

#	Document	Description

1	**SEC Registration or Charter**	SEC Certificate of Incorporation, or Republic Act/charter, establishing the certification body's legal existence and purposes.
2	**Industry / Professional Recognition**	Evidence of recognition by relevant industry or professional bodies (e.g., national association, PRC collaboration, international recognition).
3	**Documented Certification Scheme (ISO/IEC 17024-aligned)**	Written certification scheme describing competence requirements, assessment methods, recertification, and governance, aligned with ISO/IEC 17024 principles.
4	**Board Resolution — Scheme & ChainCert Approval**	Governing board resolution approving the certification scheme and authorizing issuance of credentials through ChainCert.

Others

#	Document	Description
1	**SEC or DTI Registration**	SEC Certificate of Incorporation for corporations/partnerships, or DTI Business Name Registration for sole proprietors.
2	**Board Resolution — ChainCert Authorization**	Governing board resolution approving participation in ChainCert and designating authorized officials to issue digital credentials.

Subordinate Issuer Additional Requirement

In addition to the category-specific documents above, subordinate issuers must also provide a Root Authority Authorization Letter — a formal authorization letter issued by the root authority permitting the subordinate to operate under its supervision and issue credentials through ChainCert.

Design Rationale

These document requirements replace the earlier accredited organizations database approach. Instead of relying on a static list of pre-approved organizations, the system uses document-based validation where the admin reviews actual supporting documents before granting platform access. This is more thorough because a database lookup only confirms that an organization name exists in a list — it does not verify that the person registering is authorized to represent that organization. The document review, combined with the two-tier approval gate for subordinates, provides stronger validation.

9. API Architecture

The backend exposes a collection of RESTful API endpoints grouped by domain.

Authentication Routes

Base path: /api/auth

Endpoint	Description
POST /api/auth/register	Register a new user
POST /api/auth/login	User login
POST /api/auth/change-password	Update user password
GET /api/auth/user/:id	Retrieve user profile
GET /api/auth/users	List users
POST /api/auth/update-profile	Update user profile fields
POST /api/auth/upload-pfp	Upload profile picture
POST /api/auth/forgot-password	Request password reset OTP
POST /api/auth/reset-password	Verify OTP and set new password
POST /api/auth/profile-share/generate	Generate recipient-specific share link
GET /api/auth/profile-share/:token/info	Check token validity (public)
POST /api/auth/profile-share/:token/record-view	Record view on public link
POST /api/auth/profile-share/:token/request-otp	Send OTP to recipient email
POST /api/auth/profile-share/:token/verify-otp	Verify OTP and return profile
DELETE /api/auth/profile-share/:token/revoke	Revoke a share link
POST /api/auth/profile-share/generate-public	Generate public share link (no OTP)
GET /api/auth/profile-share/my-links	Get all share links for logged-in user

Issuer Routes

Base path: /api/issuer

Endpoint	Description
POST /api/issuer/register	Submit issuer registration
POST /api/issuer/login	Issuer login
GET /api/issuer/issuer-requirements/:category	Retrieve document requirements
GET /api/issuer/:issuerId/documents	Retrieve issuer documents
GET /api/issuer/:issuerId/documents/:name	View/download issuer document
GET /api/issuer/issuer/templates/exists	Check if issuer has templates
GET /api/issuer/:issuerId/credentials	Get issuer credentials (dashboard)
GET /api/issuer/issuer/templates/:templateId/eligible-users	Get eligible users for template
GET /api/issuer/credentials/history	Get credential history (blockchain)
POST /api/issuer/credentials/create	Create credential template

Credential Routes

Base path: /api/credentials

Endpoint	Description
GET /api/credentials/templates	Retrieve credential templates
GET /api/credentials/issued	Retrieve issued credentials
GET /api/credentials/issuer/:issuerID	Get credentials by issuer ID
GET /api/credentials/:id/history	Get blockchain history of credential
POST /api/credentials/:credentialId/issue	Issue credential to holder
POST /api/credentials/:credentialId/revoke	Revoke credential
GET /api/credentials/:credentialId/verify	Verify credential
POST /api/credentials/:credentialId/log-verification	Write verification log on-chain

Admin Routes

Base path: /api/admin

Endpoint	Description
GET /api/admin/pending-issuers	Get all pending issuers
GET /api/admin/pending-issuers-admin-actionable	Get admin-actionable pending issuers
GET /api/admin/approved-issuers	Get approved issuers
GET /api/admin/rejected-issuers	Get rejected issuers
GET /api/admin/disabled-issuers	Get disabled issuers
GET /api/admin/pending-root-issuers	Get pending root issuers
GET /api/admin/pending-subordinate-issuers	Get pending subordinate issuers
GET /api/admin/pending-subordinate-awaiting-root	Get subordinates awaiting root approval
GET /api/admin/pending-subordinate-root-approved	Get root-approved subordinates
POST /api/admin/approve-issuer/:issuerId	Approve issuer application
POST /api/admin/reject-issuer/:issuerId	Reject issuer application
POST /api/admin/disable-issuer/:issuerId	Disable an issuer
POST /api/admin/enable-issuer/:issuerId	Enable an issuer
POST /api/admin/disable-user/:userId	Disable a user
POST /api/admin/enable-user/:userId	Enable a user
GET /api/admin/root/subordinates/pending	Root: pending subordinates
GET /api/admin/root/subordinates/all	Root: all subordinates
GET /api/admin/root/subordinates/root-approved	Root: approved subordinates
GET /api/admin/root/subordinates/root-rejected	Root: rejected subordinates
GET /api/admin/root/subordinates/approved	Root: fully approved subordinates
GET /api/admin/root/subordinates/disabled	Root: disabled subordinates
POST /api/admin/root-approve-subordinate/:id	Root approve subordinate
POST /api/admin/root-reject-subordinate/:id	Root reject subordinate
GET /api/admin/notifications	Get all admin notifications
GET /api/admin/notifications/unread	Get unread admin notifications

PATCH /api/admin/notifications/:id/read	Mark admin notification as read
PATCH /api/admin/notifications/read-all	Mark all admin notifications as read
GET /api/admin/credentials/all	Get all issuer credentials (admin)
POST /api/admin/trigger-expiry-check	Trigger manual expiry check
POST /api/admin/integrity-check	Trigger CouchDB vs blockchain check
POST /api/admin/change-password	Change admin password
GET /api/admin/status-logs	Get all status logs
GET /api/admin/status-logs/issuers	Get issuer status logs
GET /api/admin/status-logs/users	Get user status logs
GET /api/admin/status-logs/issuer/:email	Get status logs by issuer email
GET /api/admin/status-logs/user/:email	Get status logs by user email
GET /api/admin/issuer-decision-logs	Get all issuer decision logs
GET /api/admin/issuer-decision-logs/approved	Get issuer approval logs
GET /api/admin/issuer-decision-logs/rejected	Get issuer rejection logs
GET /api/admin/issuer-decision-logs/:email	Get decision logs by email
GET /api/admin/root-subordinate-logs	Get all root subordinate logs
GET /api/admin/root-subordinate-logs/approved	Get subordinate approval logs
GET /api/admin/root-subordinate-logs/rejected	Get subordinate rejection logs
GET /api/admin/root/subordinate-logs	Root: own subordinate logs
GET /api/admin/root/subordinate-logs/approved	Root: own approval logs
GET /api/admin/root/subordinate-logs/rejected	Root: own rejection logs

Wallet Routes

Base path: /api/wallet

Endpoint	Description
GET /api/wallet/credentials/:holderID	Retrieve wallet credentials

PATCH /api/wallet/visibility/:credentialId	Update credential visibility
PATCH /api/wallet/order/:credentialId	Update credential display order
PATCH /api/wallet/issuer-template-visibility/:id	Update issuer template visibility
GET /api/wallet/notifications/:holderID	Get user notifications
GET /api/wallet/notifications/:holderID/unread	Get unread user notifications
PATCH /api/wallet/notifications/:id/read	Mark notification as read

Verification Routes

Base path: /api/verify

Endpoint	Description
GET /api/verify/verify/:credentialId	Verify credential by URL param
POST /api/verify/verify	Verify credential by request body
GET /api/verify/verify-stats	Retrieve verification statistics
GET /api/verify/verify-credential/:credentialId	Legacy verification alias
GET /api/verify/verification-logs	Get all verification logs
GET /api/verify/verification-logs/:logId	Get verification log by ID

System Routes

Base path: /api

Endpoint	Description
GET /api/health	Health check

10. Smart Contract (Chaincode) Design

The ChainCert platform uses a Hyperledger Fabric smart contract implemented in chaincert-chaincode/index.js. The chaincode serves as the on-chain business logic layer of the system and is responsible for credential creation, issuance, revocation, expiration, verification, history retrieval, and governance logging. It ensures that critical credentials and governance events are immutably recorded on the blockchain ledger.

10.1 System Initialization

InitLedger

Initializes the chaincode. In the current implementation, this function starts the contract without inserting seed data.

10.2 Credential Template Lifecycle

CreateCredentialTemplate

Creates a credential template on-chain. A template represents the reusable blueprint of a credential before it is issued to any holder. The function validates uniqueness, assigns a deterministic blockchain timestamp, and stores the template with isTemplate: true.

10.3 Issued Credential Lifecycle

IssueCredential

Issues a credential instance to a specific holder based on an existing template. The issued credential explicitly stores templateId, holderID, issuer details, metadata, skills, issuance timestamp, and optional expiry date.

RevokeCredential

Marks an issued credential as revoked. Updates status to revoked, records a deterministic revokedAt timestamp, and stores an optional rejection reason.

ExpireCredential

Marks a credential as expired on-chain. Called by the backend expiry process when the credential's expiry date has passed.

10.4 Credential Retrieval and Verification

Get_CredentialTemplateOrIssuedCredential

Retrieves a single credential record from the ledger by exact ID (template or issued instance).

VerifyCredential

Performs on-chain verification. Checks existence, revocation, expiry, and valid status. Returns structured validity result with reason.

Check_CredentialExists

Checks whether a credential with a given ID exists on the ledger.

10.5 Credential History and Audit Trail

Get_CredentialTemplateOrIssuedCredentialHistory

Returns the full transaction history of a specific credential key with txId, timestamp, deletion flag, and recorded value.

Get_IssuerCredentialsHistory

Returns the full blockchain history of all credentials belonging to a specific issuer (templates + issued).

10.6 Bulk Query Functions

Get_HolderCredentials

Returns all credentials associated with a specific holder ID.

Get_IssuerCredentialTemplates

Returns all credential templates created by a specific issuer (isTemplate: true only).

Get_IssuerIssuedCredentials

Returns all issued, non-template credentials created by a specific issuer.

10.7 Verification Log Functions

LogVerification

Creates an immutable on-chain verification log entry with credential ID, verifier info, result, invoker identity, txId, and timestamp.

GetVerificationLogById

Retrieves one verification log entry by exact log ID.

GetAllVerificationLogs

Returns all verification log entries stored in the world state.

10.8 Issuer and User Status Log Functions

LogIssuerStatusChange

Records issuer status changes (enable, disable, etc.).

LogUserStatusChange

Records user account status changes.

GetIssuerStatusLogs

Returns status-change logs for a specific issuer email.

GetUserStatusLogs

Returns status-change logs for a specific user email.

GetAllStatusLogs

Returns all issuer and user status-change logs.

GetAllIssuerStatusLogs

Returns all issuer-only status-change logs.

GetAllUserStatusLogs

Returns all user-only status-change logs.

10.9 Issuer Decision Log Functions

LogIssuerApproval

Creates an immutable log when an administrator approves an issuer registration.

LogIssuerRejection

Creates an immutable log when an administrator rejects an issuer registration.

GetIssuerDecisionLogs

Returns all approval and rejection decision logs for a specific issuer email.

GetAllIssuerDecisionLogs

Returns all issuer decision logs across the system.

GetAllIssuerApprovalLogs

Returns all issuer approval logs only.

GetAllIssuerRejectionLogs

Returns all issuer rejection logs only.

10.10 Root Authority Subordinate Decision Log Functions

LogRootSubordinateApproval

Creates an immutable log when a root authority approves a subordinate issuer.

LogRootSubordinateRejection

Creates an immutable log when a root authority rejects a subordinate issuer.

GetAllRootSubordinateLogs

Returns all root authority subordinate decision logs.

GetAllRootSubordinateApprovalLogs

Returns all subordinate approval logs.

GetAllRootSubordinateRejectionLogs

Returns all subordinate rejection logs.

GetRootSubordinateLogsByRootAuthority

Returns subordinate decision logs for a specific root authority.

GetRootSubordinateApprovalLogsByRootAuthority

Returns subordinate approval logs for a specific root authority.

GetRootSubordinateRejectionLogsByRootAuthority

Returns subordinate rejection logs for a specific root authority.

10.11 Security and Administrative Audit Functions

LogAdminChangePassword

Records an administrator password change event on-chain.

GetAdminPasswordChangeLogs

Returns password change logs for a specific administrator email.

11. Credential Lifecycle

The lifecycle of a credential consists of several stages.

Template Creation

Issuers create credential templates defining credential name, type, description, skills, badge image, and metadata.

Credential Issuance

A credential is issued to a holder and recorded on the blockchain. The issued credential stores templateId, holderID, holderName snapshot, expiry date, and skills. A notification is created for the holder.

Credential Storage

Credential data is stored on the blockchain ledger (source of truth) while wallet caches are stored in CouchDB for fast dashboard reads.

Credential Revocation

Issuers may revoke credentials when necessary. The revocation is recorded on blockchain and synced to both CouchDB copies (issuer DB + admin wallet DB). A revocation notification is sent to the holder.

Credential Expiration

Expired credentials are automatically processed by the midnight cron job (and can be manually triggered by admin). CouchDB is updated first, then blockchain is logged.

Credential Verification

Third parties may verify credentials using credential IDs via the verification interface. Verification checks both CouchDB and blockchain, and the result is logged on-chain.

11.1 Dual-Write Architecture Pattern

All credential lifecycle operations (template creation, issuance, revocation, expiration) follow a blockchain-first, then CouchDB write order. The blockchain transaction is submitted first because it is the source of truth. If the blockchain write succeeds but CouchDB fails, the system returns a partial success response with a warning rather than failing entirely. This ensures the immutable record is preserved even when the operational layer encounters an issue. The admin integrity check feature can detect these mismatches by comparing CouchDB records against blockchain records field by field.

For credential issuance specifically, the CouchDB write involves multiple sequential operations: saving to the issuer's CouchDB (issued_credentials), saving to the admin CouchDB (wallet_user_credentials), and creating a holder notification (user_notifications). These are not wrapped in a multi-database transaction because CouchDB does not support cross-database transactions.

Section 11.1: Adding Fabric Identities Under an Existing Organization

When multiple issuer accounts need to operate under the same organization node (e.g., multiple subordinate issuers under DLSU), each account requires its own Fabric identity. This involves two steps: registering and enrolling the identity in the organization's Certificate Authority, then importing it into the application wallet.

Step 1: Register the Identity in the Organization's CA

Edit the organization's registerEnroll script (e.g., registerEnrollFORDLSU.sh) to add a new identity registration block. The identity name and secret must be unique within the CA.

```
fabric-ca-client register \
  --caname ca-dlsu \
  --id.name <IDENTITY_NAME> \
  --id.secret <IDENTITY_SECRET> \
  --id.type client \
  --tls.certfiles "${PWD}/../../organizations/fabric-ca/dlsu/tls-cert.pem"
```

For example, to register a new subordinate called "lasallecomputersociety":

```
fabric-ca-client register \
  --caname ca-dlsu \
  --id.name dlsusubordinatelasallecomputersociety \
  --id.secret dlsusubordinatelasallecomputersocietypw \
  --id.type client \
  --tls.certfiles "${PWD}/../../organizations/fabric-ca/dlsu/tls-cert.pem"
```

Step 2: Enroll the Identity to Generate MSP Materials

Enroll the registered identity to generate its certificate and private key. The output directory follows the convention: users/<IdentityLabel>@dlsu.example.com/msp.

```
fabric-ca-client enroll \
 -u https://<IDENTITY_NAME>:<IDENTITY_SECRET>@localhost:12054 \
 --caname ca-dlsu \
 -M
 "${PWD}/../../organizations/peerOrganizations/dlsu.example.com/users/<IdentityLabel>@dlsu.example.com/msp" \
 --tls.certfiles "${PWD}/../../organizations/fabric-ca/dlsu/tls-cert.pem"
```

Step 3: Apply NodeOU Configuration

Copy the organization's NodeOU config into the new identity's MSP directory. This is required for Fabric to recognize the identity's role.

```
cp "${PWD}/../../organizations/peerOrganizations/dlsu.example.com/msp/config.yaml" \

"${PWD}/../../organizations/peerOrganizations/dlsu.example.com/users/<IdentityLabel>@dlsu.example.com/msp/config.yaml"
```

Step 4: Import the Identity into the Application Wallet

Create or edit a wallet setup script (based on setupWallet_DLSU.js) to import the new identity. The script reads the certificate and private key from the enrolled MSP directory and writes them to the application wallet with a unique label.

Key fields in the wallet setup script:

```
const credPath = path.join(
 process.env.HOME,

'fabric-samples/fabric-samples/test-network/organizations/peerOrganizations/dlsu.example.com/users/<IdentityLabel>@dlsu.example.com/msp'
);

const identity = {
 credentials: { certificate, privateKey },
 mspId: 'DLSUMSP',
 type: 'X.509',
};
```

```
await wallet.put('<wallet-identity-label>', identity);
```

The wallet identity label (e.g., "delasalleuniversity-subordinate-lasallecomputersociety") is the value stored in the issuer's fabricIdentity field in CouchDB and is used by the backend to resolve which Fabric identity to use when submitting transactions on behalf of that issuer.

Run the script from the backend directory:

```
cd ~/chaincert-api
node setup_Wallet_DLSU_Subordinate_LASALLECOMPUTERSOCIETY.js
```

Step 5: Verify the Wallet

After running the setup script, confirm the identity exists in the wallet directory:

```
ls ~/chaincert-api/wallet/
```

Expected output should include a file named after the identity label (e.g., delasalleuniversity-subordinate-lasallecomputersociety.id).

Important Notes

All subordinate identities under the same root authority share the same mspId (e.g., DLSUMSP). They are distinct identities within the same organizational trust domain — not separate organizations. The identity type for subordinate issuers is "client" (not "admin" or "peer"), because they only need to submit transactions, not administer the peer or the channel. No new peer node is deployed for subordinate issuers.

12. Database Design

ChainCert uses CouchDB as its off-chain database layer, with multiple databases distributed across the main administrative database instance and issuer-specific database instances.

12.1 Main Administrative CouchDB Instance (Port 5985)

This instance contains the following databases:

users

Stores platform user accounts, profile details, authentication info, role assignments, and account status.

issuers

Stores issuer registration records, onboarding details, approval status, issuer type, supporting documents (as CouchDB attachments), Fabric identity metadata, and governance info.

user_notifications

Stores user notification entries including credential issuance and revocation notifications with read/unread status.

admin_notifications

Stores admin dashboard notification events triggered by issuer registration, root subordinate decisions, and admin approval/rejection actions.

couchdb_connections

Stores connection metadata mapping issuer orgNames to their CouchDB URLs for dynamic database resolution.

profile_share_links

Stores generated profile-sharing tokens, recipient email info, expiration settings, OTP fields, view counts, and active/revoked states.

wallet_user_credentials

Stores wallet-facing off-chain copies of credentials assigned to holders. Supports dashboard rendering, wallet views, credential visibility settings, ordering preferences, and integrity checking.

12.2 Issuer CouchDB Instances

Each approved issuer organization receives its own dedicated CouchDB instance:

Issuer	Port	Role
DLSU (De La Salle University)	9084	Root authority issuer
INNOCENCIO	9085	Subordinate issuer
La Salle Computer Society	9086	Subordinate issuer

Each issuer instance contains the following databases:

issued_credentials

Stores issued credential instances associated with the issuer. Supports issuer-side fast reads, dashboard display, and lifecycle sync with blockchain.

wallet_issuer_credentials

Stores issuer-side credential templates and related credential cache records used for template retrieval, issuance preparation, visibility settings, and credential management views.

12.3 Database Design Rationale

The database architecture separates on-chain truth from off-chain operational storage. Hyperledger Fabric stores the immutable source-of-truth records for credential lifecycle and governance events, while CouchDB stores: user and issuer account data, uploaded onboarding documents, notification records (user and admin), wallet-friendly credential views, profile-sharing data, and issuer-local operational credential caches.

This design improves: frontend response speed, dashboard query efficiency, separation of administrative and issuer-owned data, and maintainability of operational records without compromising ledger integrity.

13. Security Model

Security mechanisms include:

Identity Security

Hyperledger Fabric identities use X.509 certificates issued by Fabric Certificate Authorities.

Authentication

API authentication uses JWT tokens. Tokens are attached via the Authorization header and decoded by Express middleware on every request.

Password Security

Passwords are hashed using bcrypt (bcryptjs). Forgot-password flow uses time-limited OTP codes sent via email.

Ledger Integrity

Hyperledger Fabric ensures immutable ledger storage. The admin can trigger a CouchDB vs blockchain integrity check that performs field-by-field comparison of credential records.

Audit Logging

All major governance actions (issuer approval/rejection, user enable/disable, subordinate decisions, admin password changes, credential verifications) are recorded on the blockchain.

Email Validation

Email validation uses regex on registration to prevent invalid email addresses.

14. System Directory Structure

The ChainCert deployment environment is organized across backend, chaincode, and Hyperledger Fabric network directories.

~/chaincert-api/

Main backend application directory.

~/chaincert-api/sdk-server-fixed.js

Express server entry point with Fabric gateway helper, middleware, and route mounting.

~/chaincert-api/couchDB_service.js

Centralized CouchDB data access layer (users, issuers, wallets, credentials, notifications, share links).

~/chaincert-api/.env

Environment configuration (JWT_SECRET, etc.).

~/chaincert-api/controllers/

Backend business logic controllers (authController, accountController, issuerOnboardingController, issuerCredentialController, adminIssuerController, adminAuditController, adminDashboardController, rootAuthorityController, verifierController, walletController).

~/chaincert-api/routes/

REST API route definitions (authRoutes, adminRoutes, issuerRoutes, credentialRoutes, walletRoutes, verificationRoutes, systemRoutes).

~/chaincert-api/config/

Backend configuration files (issuerRequirements.js).

~/chaincert-api/wallet/

Enrolled Fabric wallet identities (chaincert-admin, delasalleuniversity-admin, delasalleuniversity-subordinate-innocencio, delasalleuniversity-subordinate-lasallecomputersociety).

~/chaincert-chaincode/

Hyperledger Fabric smart contract source (index.js, package.json).

~/fabric-samples/.../test-network/

Main Hyperledger Fabric test-network directory.

~/fabric-samples/.../test-network/addDLSU/

DLSU organization onboarding assets (connection profiles, CA scripts, Docker Compose files).

~/fabric-samples/.../test-network/addDLSU/compose/

Docker Compose definitions for DLSU peer, CA, and CouchDB services.

~/fabric-samples/.../test-network/channel-artifacts/

Generated channel artifacts (chaincert-channel.block, config blocks, update envelopes).

~/fabric-samples/.../test-network/configtx/

Channel configuration files including configtx.yaml.

~/fabric-samples/.../test-network/organizations/

Generated cryptographic materials, MSP directories, TLS certificates, connection profiles.

~/fabric-samples/.../test-network/organizations/peerOrganizations/

Peer org assets for org1.example.com, org2.example.com, and dlsu.example.com.

Directory Structure Notes

The structure reflects three major deployment domains: Application Layer (backend API + chaincode), Blockchain Network Layer (Fabric test network, org assets, channel artifacts, scripts, crypto materials), and Identity/Configuration Layer (wallet identities, MSP directories, TLS certificates, Fabric CA records, connection profiles).

Section 14.1: Backend Dependencies

The ChainCert backend requires the following npm packages, installed via npm install in the ~/chaincert-api directory.

Package	Purpose
express	Web framework for the REST API server
cors	Enables cross-origin requests between frontend (port 3000) and backend (port 4000)
dotenv	Loads environment variables from .env configuration file
bcryptjs	Secure password hashing using bcrypt (10 rounds)

jsonwebtoken	JWT token generation and verification for API authentication
multer	Handles multipart file uploads (issuer document submissions)
nodemailer	Email delivery for OTP codes (forgot password, share link verification)
node-cron	Scheduled job execution (midnight credential expiry check)
dockerode	Docker API client for container management
fabric-network	Hyperledger Fabric SDK for gateway connections and transaction submission
fabric-ca-client	Fabric Certificate Authority client for identity enrollment
nano	CouchDB client library for database operations

Frontend Dependencies (installed in the React project directory):

Package	Purpose
jspdf	Generates downloadable PDF files for credential export
qrcode	Generates QR codes embedded in exported credential PDFs

Chaincode Dependencies (installed in ~/chaincert-chaincode):

Package	Purpose
fabric-contract-api	Fabric smart contract API for implementing chaincode logic
fabric-shim	Fabric chaincode shim for runtime interaction with the peer

15. System Limitations

The following are known limitations, trade-offs, and areas for improvement that the team acknowledges. These are not oversights but conscious design decisions within the capstone scope, and each has a prepared design rationale.

Governance & Trust

- In the current desig implemented, the administrator remains a single point of failure for governance — can approve fake issuers, but every action is immutably logged on blockchain (detection, not prevention).
- Governance is centralized by design — appropriate for a platform model, but distributed governance is architecturally supported via Fabric's configtx.yaml policy mechanism.
- Garbage-in risk — blockchain makes data immutable but does not guarantee legitimacy of data entering the system. Exists across all three deployment designs.
- Onboarding legitimacy depends on admin validation quality — document review is a human process.

Security

- JWT secret has a hardcoded fallback ('dev_secret_key') if .env is missing. Production: remove fallback.
- CouchDB credentials hardcoded as admin:adminpw across all files. Production: use environment variables.
- JWT tokens stored in localStorage (XSS-vulnerable). Production: migrate to httpOnly cookies.
- No rate limiting on any endpoint. Production: add express-rate-limit.
- All traffic over HTTP, not HTTPS. Production: add TLS termination.
- Disabled users' JWT tokens remain valid up to 24 hours (no token blacklist).
- Shared Fabric admin identity — no per-admin cryptographic attribution. Individual accountability via logged metadata.

Architecture

- Single orderer node — single point of failure. Production: Raft consensus cluster with 3-5 orderers.
- All infrastructure on one machine — 3 ledger copies on same server don't provide genuine infrastructure redundancy.
- Mutable CouchDB read layer — mitigated by blockchain verification + integrity check, but UI could display tampered data between integrity checks.
- One-directional integrity check (CouchDB → blockchain only) — orphaned blockchain records with missing CouchDB copies would not be detected.

Data & Operations

- Dual-write pattern with no multi-database transaction support — partial failures handled by warnings, not automatic rollback.
- Credential expiry has a timing gap — credentials remain 'valid' on blockchain until the cron job or admin manual trigger runs. No real-time expiry check in VerifyCredential chaincode.
- No notification on credential expiry — holder's credential silently becomes expired.
- Duplicate prevention relies on CouchDB, not blockchain-level check — partial write failure could allow duplicate issuance.
- Credential IDs use Date.now() + Math.random() — not cryptographically secure, but ID is not a security token.
- No formal throughput benchmarking performed.

Features Not Implemented

- Private Data Collections (selective visibility between orgs on same channel).
- Distributed channel governance (multi-org signing for config updates).
- Automated node provisioning (requires tools like Hyperledger Bevel).
- Blockchain-to-CouchDB reverse integrity scan.
- Cron-based CouchDB-blockchain synchronization.

16. Access Control Model

The ChainCert platform enforces a multi-layered access control model across the API, blockchain, and database layers to ensure that only authorized users and organizations can perform specific actions.

16.1 API-Level Access Control (JWT-Based RBAC)

The backend uses JSON Web Tokens (JWT) to authenticate requests. Each request: extracts the token from the Authorization header, verifies the token via the middleware in sdk-server-fixed.js, and attaches user data to req.user.

Role-Based Access Control (RBAC)

Role	Access Scope
Administrator	Full system control (issuer governance, user management, integrity checks, notifications, credentials view, status logs, password management)
Issuer	Credential operations only (template creation, issuance, revocation, history, eligible users, template visibility, share profile)
User / Holder	Wallet + profile only (credentials, visibility, ordering, notifications, share links)
Verifier	Verification-only access (verify credential, log verification event, view verification stats)

Controllers explicitly validate roles before execution. Issuer identity is further resolved via CouchDB (approved issuer lookup by email) before blockchain transactions.

16.2 Blockchain-Level Access Control

Hyperledger Fabric enforces identity through X.509 certificates, MSP validation, and transaction signing. Each transaction is tied to a verified organization identity. The backend resolves the correct Fabric identity (chaincert-admin, delasalleuniversity-admin, or subordinate identity) based on the user's role and issuer record.

16.3 Database-Level Access Control

CouchDB is separated per organization:

- Port 5985 → Admin/global data (users, issuers, notifications, wallet credentials, share links)

- Port 9084 → DLSU root authority issuer data
- Port 9085 → INNOCENCIO subordinate issuer data
- Port 9086 → La Salle Computer Society subordinate issuer data

This ensures data isolation, organization-level separation, and security boundary enforcement.

17. On-Chain vs Off-Chain Data Design

The system follows a hybrid architecture.

Layer	Data Stored	Purpose
On-Chain (Blockchain)	Credentials, status changes, governance logs, verification records, admin audit logs	Immutability, auditability, tamper resistance
Off-Chain (CouchDB)	Users, issuers, documents, wallet data, notifications (user + admin), share links, credential caches	Performance, flexibility, fast dashboard reads

Design Principle

Blockchain = source of truth for credential integrity and governance decisions. CouchDB = operational layer for application state, UI rendering, and fast reads. The admin integrity check feature validates that CouchDB records match blockchain records field by field.

18. Key Transaction Flows

18.1 Credential Issuance

- Issuer triggers issuance from frontend dashboard
- Backend validates JWT and resolves issuer Fabric identity
- Controller checks issuer approval status and CouchDB URL
- Fabric SDK submits IssueCredential transaction to blockchain
- Chaincode creates issued credential record on ledger
- Backend enriches credential with holderName, badgeImage, skills from template
- Credential copy saved to issuer CouchDB (issued_credentials)
- Credential copy saved to admin CouchDB (wallet_user_credentials)
- Holder notification created in user_notifications
- Response returned to frontend

18.2 Credential Verification

- Verifier inputs credential ID
- Backend checks CouchDB (wallet_user_credentials) for fast lookup
- Blockchain verification executed via VerifyCredential chaincode
- Result computed (valid/revoked/expired with reason)
- Verification log recorded on-chain via LogVerification

18.3 Issuer Onboarding

- Issuer submits registration with documents (stored as CouchDB attachments)
- If subordinate: root authority reviews and approves/rejects first (logged on blockchain)
- Admin reviews application: If approved is: (1) root authority, perform node deployment for that issuer, (2) subordinate, create a fabric identity for it.
- Admin approval assigns Fabric identity, CouchDB URL, and wallet ID
- Decision logged on blockchain (LogIssuerApproval / LogIssuerRejection)
- Admin notification created

- Issuer status updated in CouchDB

18.4 Credential Revocation

- Issuer triggers revocation with optional rejection reason
- Backend resolves Fabric identity and submits RevokeCredential to blockchain
- CouchDB copies updated in both issuer DB and admin wallet DB
- Revocation notification sent to holder

18.5 Profile Sharing

- User generates a share link (public or recipient-specific)
- Link stored in profile_share_links with token, expiry, and optional recipient email
- Recipient accesses link; if recipient-specific, OTP is sent via email
- After OTP verification (or direct access for public links), profile data is returned
- View count incremented on each access

19. Scalability Considerations

Current State

- Single channel (chaincert-channel)
- Multiple organization nodes (Org1, Org2, DLSU)
- Separate CouchDB instances per issuer
- Containerized deployment using Docker Compose

Future Improvements

Multi-Channel Architecture

The system currently uses a single blockchain channel. A multi-channel architecture becomes relevant when different organizations require data isolation, sensitive transactions must not be visible to all participants, or network traffic increases significantly. For example, if ChainCert is adopted nationwide: Education sector → Channel A, Government certifications → Channel B, Private training institutions → Channel C.

Load-Balanced Backend

The backend currently runs as a single service instance. Load balancing becomes necessary when multiple users perform actions simultaneously (e.g., mass credential issuance during graduation), high-frequency verification requests occur, or system uptime becomes critical.

CouchDB Clustering

Currently, each CouchDB instance runs as a standalone database per organization. Clustering becomes relevant when database size grows significantly, high read/write operations occur, or high availability is required. CouchDB clustering ensures data replication across nodes, no single point of failure, and faster query performance.

Performance Design

Blockchain → integrity (source of truth, immutable records). CouchDB → speed (fast dashboard reads, operational queries). The current use of Docker-based deployment simplifies replication during development, but large-scale production deployment would require stronger orchestration and monitoring mechanisms.

19.1 Deployment Architecture Extensions

The ChainCert platform supports three deployment architecture designs. The choice of design affects who operates infrastructure and who participates in governance decisions, but does not change the application code, chaincode, or frontend.

Design 1 — Platform (Current)

ChainCert manages all infrastructure and governance. Issuers use the platform as a service. All peers, orderers, CAs, and CouchDB instances are hosted on a single ChainCert-operated server. The configtx.yaml Admins policy is set to OR(Org1MSP.admin), meaning ChainCert alone signs all channel configuration updates.

Design 1 process: ChainCert performs all phases on their server — deploy CA container (compose-ca-dlsu.yaml, port 12054), register and enroll identities (registerEnrollFORDLSU.sh), deploy peer container (compose-dlsu.yaml, port 12051, CORE_PEER_LOCALMSPID=DLSUMSP), deploy CouchDB (compose-couch-dlsu.yaml, port 9084), update channel config (configtx-WITHDLSU.yaml adds DLSU org definition), join peer to channel, install and approve chaincode, import wallet identity (setupWallet_DLSU.js), create CouchDB databases, register couchdb_connections mapping, approve in UI.

Advantages: Fastest onboarding, simplest operations, lowest barrier, cheapest.

Limitations: Single point of compromise, no real data sovereignty, scalability ceiling.

Design 2 — Hybrid

Issuers deploy their own nodes for data sovereignty, but ChainCert retains sole governance authority. The configtx.yaml Admins policy stays OR(Org1MSP.admin). Node ownership is separated from governance.

Design 2 process: The issuer performs infrastructure phases on their own server using a deployment kit provided by ChainCert. The issuer sends back: MSP definition, peer endpoint, peer TLS certificate, and admin certificate. ChainCert adds the org to configtx.yaml, generates and signs the channel config update, and submits it. Key backend change: connection profile peer URL changes from localhost to the issuer's server address, and gateway discovery asLocalhost changes from true to false.

Advantages: Genuine data sovereignty, genuine infrastructure redundancy, satisfies government requirements, simple governance.

Limitations: Higher onboarding complexity, requires deployment kit and support.

Design 3 — Full Consortium

Issuers deploy their own nodes AND participate in governance. The configtx.yaml Admins policy changes to MAJORITY(all member MSPs). New member admission requires cryptographic signatures from a majority of existing members.

Design 3 process: Infrastructure identical to Design 2. The only difference is channel config update signing. The signed envelope must be passed to each existing member org, each runs peer channel signconfigtx, and once a majority signs, the envelope is submitted. This requires a coordination mechanism and governance workflow.

Advantages: Maximum decentralization, strongest trust model.

Limitations: Slowest onboarding, governance complexity, highest cost, only makes sense for equal-stake institutional consortiums.

Configuration Control Point

The configtx.yaml Admins policy is the single configuration point that determines whether the setup is Design 1, 2, or 3. Changing it from OR(Org1MSP.admin) to MAJORITY(all member MSPs) transitions governance from centralized to distributed without changing application code, chaincode, or frontend.

Transition Paths

Design 1 → Design 2: Issuer deploys own peer/CA/CouchDB using deployment kit. Connection profiles updated. No governance policy change needed.

Design 2 → Design 3: Edit configtx.yaml Admins policy from OR to MAJORITY. Implement multi-org signing workflow for channel updates.

What Stays the Same Across All Designs

- Chaincode (smart contract logic)
- Frontend application
- API endpoints and controller logic
- CouchDB database schema
- Credential lifecycle (issuance, revocation, verification, expiry)
- Notification system
- Admin integrity check mechanism

Design Selection Rationale

Design 1 was chosen because:

- (a) the core value proposition (blockchain-backed credential issuance, verification, auditability) functions identically regardless of who hosts infrastructure
- (b) Design 2 introduces infrastructure distribution complexity outside capstone scope
- (c) the prototype operates on a single machine with no actual external institutions so Design 2 provides zero real data sovereignty
- (d) UAT testers would need to deploy their own Fabric nodes before testing — making UAT impractical
- (e) the system demonstrates extensibility through documented Fabric policy mechanisms without implementing them.

20. Testing Strategy

The ChainCert prototype employs a manual testing strategy focused on validating end-to-end workflows across all system roles.

Functional Testing

Each feature is tested through the web interface by executing the full user flow: credential template creation, issuance, revocation, verification, profile sharing, notifications, and admin governance actions.

Blockchain Verification

On-chain records are verified using Fabric CLI peer chaincode query commands (documented in Appendix A) to confirm that blockchain state matches expected outcomes after each operation.

Integrity Checking

The admin integrity check feature provides automated validation that CouchDB credential records match blockchain records field by field, covering: credentialName, credentialType, holderID, status, issuedDate, expiryDate, description, and issuerId.

Database Validation

CouchDB records are inspected via the Fauxton UI (port 5985/_utils for admin DB, port 9084/_utils for DLSU, etc.) to verify correct data persistence after operations.

References:

3D-Universal. (2025, October 28). University accreditation and CHED recognition explained. Retrieved from https://3d-universal.com/en/blogs/university-accreditation-and-ched-recognition-explained.html

Commission on Higher Education. (2021, April 5). Verification of status of higher education institutions and their programs. Retrieved from https://ieducationphl.ched.gov.ph/ncaim/verification-of-status-of-higher-education-institutions-and-their-programs/

Philippine Embassy in Washington, D.C. (2024, June 30). Verification of status of higher education institutions and their programs. Retrieved from https://philippineembassy-dc.org/verification-of-status-of-higher-education-institutions-and-their-programs/

Philippine Embassy in Seoul. (2025, December 22). Advisory: Verification of higher education institutions. Retrieved from https://www.philembassy-seoul.com/announcements-advisories/advisory-verification-of-higher-education-institutions

Wise. (2025, November 18). SEC business registration in the Philippines: Complete guide. Retrieved from https://wise.com/ph/blog/sec-business-registration

FilePino. (2025, April 14). SEC registration: How to register a corporation in the Philippines. Retrieved from https://www.filepino.com/how-to-register-a-corporation-in-the-philippines/

Respicio & Co. Law Firm. (2025, October 2). How to verify if a Philippine government agency or department is legitimate. Retrieved from https://www.respicio.ph/commentaries/how-to-verify-if-a-philippine-government-agency-or-department-is-legitimate

Executive Order No. 122-C. (1987). Creating the Office for [example of agency creation by EO]. Retrieved from https://elibrary.judiciary.gov.ph/thebookshelf/showdocs/5/7420

Technical Education and Skills Development Authority. (2013, April 30). Program registration and accreditation (UTPRAS). Retrieved from https://www.tesda.gov.ph/About/TESDA/26

TESDA Region XII. (2025, November 23). Program registration and accreditation. Retrieved from https://sites.google.com/tesda.gov.ph/tesdaregion12/programs-and-services/program-registration-and-accreditation

TESDA Region V – Sorsogon. (2025, June 12). Unified TVET Program Registration and Accreditation System. Retrieved from http://tesda5.com.ph/sorsogon/programs-and-services/

Wise. (2025, November 18). SEC business registration in the Philippines: Complete guide. Retrieved from https://wise.com/ph/blog/sec-business-registration

First Circle. (2025, April 22). How to register your business name online at BNRS DTI. Retrieved from https://www.firstcircle.ph/blog/business-name-registration-bnrs-dti

Kittelson & Carpo. (2022, June 22). Department of Trade and Industry – DTI registration in the Philippines. Retrieved from https://kittelsoncarpo.com/business-registration/government-agencies/dti/

International Organization for Standardization. (2012). ISO/IEC 17024:2012 – Conformity assessment – General requirements for bodies operating certification of persons. Retrieved from https://cdn.standards.iteh.ai/samples/52993/ddec6c8759214c5d958bf8f6d6d9ffaf/ISO-IEC-17024-2012.pdf

International Organization for Standardization. (n.d.). How to develop schemes for the certification of persons. Retrieved from

https://www.iso.org/iso/casco_-_certification_of_persons.pdf

National Accreditation Board (INAB). (n.d.). ISO/IEC 17024 – Conformity assessment – General requirements for bodies operating certification of persons [Information note]. Retrieved from https://www.inab.ie/news-resources/publications/iso-iec-17024-conformity-assessment-general-requirements-for-bodies-operating-ce

3D-Universal. (2025, October 28). University accreditation and CHED recognition explained. Retrieved from https://3d-universal.com/en/blogs/university-accreditation-and-ched-recognition-explained.html

Wise. (2025, November 18). SEC business registration in the Philippines: Complete guide. Retrieved from https://wise.com/ph/blog/sec-business-registration

FilePino. (2025, April 14). SEC registration: How to register a corporation in the Philippines. Retrieved from https://www.filepino.com/how-to-register-a-corporation-in-the-philippines/

Appendix A — Hyperledger Fabric Chaincode CLI Command Reference

This appendix presents the Hyperledger Fabric CLI commands used to interact directly with the chaincert smart contract deployed on the chaincert-channel. Each command includes a template version (with placeholders), an actual command (with realistic example values), and the expected output.

1. Prerequisites

Run these commands once per terminal session before executing any peer chaincode queries or invokes.

```
# 1) Navigate to the Fabric test network directory
cd ~/fabric-samples/fabric-samples/test-network

# 2) Export Fabric binaries path
export PATH=${PWD}/../bin:$PATH
```

```
# 3) Export Fabric configuration path
export FABRIC_CFG_PATH=${PWD}/../config/

# 4) Use the DLSU peer context
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="DLSUMSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.exa
mple.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/dlsu.example.com/users/Admin@dlsu.exam
ple.com/msp
export CORE_PEER_ADDRESS=localhost:12051

# 5) Optional: common variables for invoke commands
ORDERER_CA="${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/
tlsca.example.com-cert.pem"
ORG1_PEER_TLS="${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
"
DLSU_PEER_TLS="${PWD}/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/ca.crt"
```

2. GET Operations (Read / Query)

2.1 Get Single Credential

Get a specific credential (template or issued) by its exact ID.

Template Command (with placeholders)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredential","CREDENTIAL_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredential","cred_1710000000000_abc123def"]}'
```

Expected Output

```
{"id":"cred_1710000000000_abc123def","issuerID":"issuer_dlsu_admin_com_1710000000000",
```

```
"credentialType":"Academic","credentialName":"Bachelor of Science in IT",
"status":"issued","holderID":"user123","isTemplate":false}
```


2.2 Get Credential Templates (by Issuer)

Get all credential templates created by a specific issuer.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["Get_IssuerCredentialTemplates","ISSUER_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["Get_IssuerCredentialTemplates","issuer_dlsu_admin_com_1710000000000"]}'
```

Expected Output


```
[{"id":"cred_1710000000000_abc123def","credentialName":"Bachelor of Science in IT",
"credentialType":"Academic","isTemplate":true,"status":"active"}, ...]
```


2.3 Get Issued Credentials (by Issuer)

Get all issued (non-template) credentials under a specific issuer.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["Get_IssuerIssuedCredentials","ISSUER_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["Get_IssuerIssuedCredentials","issuer_dlsu_admin_com_1710000000000"]}'
```

Expected Output


```
[{"id":"cred_1710000000000_abc123def_user123_1710000100000",
"holderID":"user123","status":"issued","credentialName":"BSIT Diploma"}, ...]
```

2.4 Get Holder Credentials

Get all credentials issued to a specific holder.

Template Command (with placeholders)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_HolderCredentials","HOLDER_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_HolderCredentials","abc123def456ghi789"]}'
```

Expected Output

```
[{"id":"cred_...","credentialName":"BSIT Diploma","status":"issued",
"holderID":"abc123def456ghi789","issuerID":"issuer_dlsu_..."}, ...]
```

2.5 Get Credential History

Get full history / audit trail of a specific credential.

Template Command (with placeholders)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredentialHistory","CREDENTIAL_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredentialHistory","cred_1710000000000_abc123def"]}'
```

Expected Output

```
[{"txId":"a1b2c3d4...","timestamp":"2026-03-01T10:00:00Z","isDelete":false,
"value":{"status":"active","credentialName":"BSIT Diploma",...}},
{"txId":"e5f6g7h8...","timestamp":"2026-03-15T14:30:00Z","isDelete":false,
"value":{"status":"issued","holderID":"user123",...}}]
```

2.6 Check Credential Exists

Check if a credential exists on the ledger (returns true/false).

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["Check_CredentialExists","CREDENTIAL_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["Check_CredentialExists","cred_1710000000000_abc123def"]}'
```

Expected Output

```
true
```

2.7 Verify Credential

Verify if a credential is valid (checks existence, revocation, expiry, status).

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["VerifyCredential","CREDENTIAL_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["VerifyCredential","cred_1710000000000_abc123def_user123_1710000100000"]}'
```

Expected Output

```
{"credentialId":"cred_...","isValid":true,"status":"issued",
"isRevoked":false,"isExpired":false,"reason":"Credential is valid"}
```

2.8 Get Issuer Credentials History

Get the full blockchain audit trail of all credentials (templates + issued) under a specific issuer.

Template Command (with placeholders)

```
peer chaincode query \
```

```
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_IssuerCredentialsHistory","ISSUER_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_IssuerCredentialsHistory","issuer_dlsu_admin_com_1710000000000"]}'
```

Expected Output

```
[{"credentialId":"cred_...","credentialName":"BSIT Diploma",
"history":[{"txId":"...","timestamp":"...","value":{...}}]}, ...]
```

2.9 Get All Verification Logs

Get all verification log entries stored in the world state.

Template Command (with placeholders)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllVerificationLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllVerificationLogs"]}'
```

Expected Output

```
[{"logId":"vlog_a1b2c3...","credentialId":"cred_...",
"verifierName":"HR Department","verifierEmail":"hr@company.com",
"result":{"valid":true,"reason":"ok"},"timestamp":"2026-03-20T09:00:00Z"}, ...]
```

2.10 Get Verification Log by ID

Get a single verification log entry by its ID.

Template Command (with placeholders)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
```

```
-c '{"Args":["GetVerificationLogById","VERIFICATION_LOG_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetVerificationLogById","vlog_a1b2c3d4e5f6g7h8"]}'
```

Expected Output

```
{"logId":"vlog_a1b2c3d4e5f6g7h8","credentialId":"cred_...",
"verifierName":"HR Department","result":{"valid":true,"reason":"ok"}}
```

2.11 Get Issuer Status Logs

Get all status-change logs for a specific issuer email.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetIssuerStatusLogs","ISSUER_EMAIL"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetIssuerStatusLogs","dlsu@email.com"]}'
```

Expected Output

```
[{"email":"dlsu@email.com","previousStatus":"approved","newStatus":"disabled",
"reason":"Compliance review","timestamp":"2026-03-10T12:00:00Z"}, ...]
```

2.12 Get User Status Logs

Get all status-change logs for a specific user email.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetUserStatusLogs","USER_EMAIL"]}'
```

Actual Command (with example values)

```
peer chaincode query \
```

```
-C chaincert-channel \
-n chaincert \
-c '{"Args":["GetUserStatusLogs","student@email.com"]}'
```

Expected Output

```
[{"email":"student@email.com","previousStatus":"active","newStatus":"disabled",
"reason":"Account violation","timestamp":"2026-03-12T08:00:00Z"}, ...]
```

2.13 Get All Status Logs

Get all issuer + user status-change logs.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllStatusLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllStatusLogs"]}'
```

Expected Output

```
[{"type":"issuer_status_log","email":"dlsu@email.com",...},
{"type":"user_status_log","email":"student@email.com",...}, ...]
```

2.14 Get All Issuer Status Logs

Get all issuer-only status-change logs.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllIssuerStatusLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllIssuerStatusLogs"]}'
```

Expected Output

```
[{"type":"issuer_status_log","email":"dlsu@email.com",
"previousStatus":"approved","newStatus":"disabled",...}, ...]
```

2.15 Get All User Status Logs

Get all user-only status-change logs.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllUserStatusLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllUserStatusLogs"]}'
```

Expected Output

```
[{"type":"user_status_log","email":"student@email.com",...}, ...]
```

2.16 Get Issuer Decision Logs (by Email)

Get issuer approval/rejection logs for a specific issuer email.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetIssuerDecisionLogs","ISSUER_EMAIL"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetIssuerDecisionLogs","dlsu@email.com"]}'
```

Expected Output

```
[{"type":"issuer_approval","issuerEmail":"dlsu@email.com",
"orgName":"De La Salle University","timestamp":"2026-02-15T10:00:00Z"}, ...]
```

2.17 Get All Issuer Decision Logs

Get all issuer decision logs.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllIssuerDecisionLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllIssuerDecisionLogs"]}'
```

Expected Output

```
[{"type":"issuer_approval",...}, {"type":"issuer_rejection",...}, ...]
```

2.18 Get All Issuer Approval Logs

Get only issuer approval logs.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllIssuerApprovalLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllIssuerApprovalLogs"]}'
```

Expected Output

```
[{"type":"issuer_approval","issuerEmail":"dlsu@email.com","orgName":"DLSU",...}, ...]
```

2.19 Get All Issuer Rejection Logs

Get only issuer rejection logs.

Template Command (with placeholders)

```
peer chaincode query \
```

```
-C chaincert-channel \
-n chaincert \
-c '{"Args":["GetAllIssuerRejectionLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllIssuerRejectionLogs"]}'
```

Expected Output

```
[{"type":"issuer_rejection","issuerEmail":"fake@org.com","reason":"Incomplete docs",...}, ...]
```

2.20 Get All Root Subordinate Logs

Get all root subordinate decision logs.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllRootSubordinateLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllRootSubordinateLogs"]}'
```

Expected Output

```
[{"type":"root_subordinate_approval","orgName":"INNOCENCIO",...},
{"type":"root_subordinate_approval","orgName":"La Salle Computer Society",...}, ...]
```

2.21 Get All Root Subordinate Approval Logs

Get all subordinate approval logs only.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllRootSubordinateApprovalLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllRootSubordinateApprovalLogs"]}'
```

Expected Output

```
[{"type":"root_subordinate_approval","orgName":"INNOCENCIO",...}, ...]
```

2.22 Get All Root Subordinate Rejection Logs

Get all subordinate rejection logs only.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllRootSubordinateRejectionLogs"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetAllRootSubordinateRejectionLogs"]}'
```

Expected Output

```
[{"type":"root_subordinate_rejection",...}, ...]
```

2.23 Get Root Subordinate Logs by Root Authority

Get subordinate logs for a specific root authority.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetRootSubordinateLogsByRootAuthority","ROOT_AUTHORITY_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetRootSubordinateLogsByRootAuthority","issuer_dlsu_admin_com_1710000000000"]}'
```

Expected Output

```
[{"rootAuthorityId":"issuer_dlsu_admin_com_1710000000000",
"orgName":"INNOCENCIO","type":"root_subordinate_approval",...}, ...]
```

2.24 Get Root Sub Approval Logs by Root Authority

Get approved-only subordinate logs for a specific root authority.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetRootSubordinateApprovalLogsByRootAuthority","ROOT_AUTHORITY_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetRootSubordinateApprovalLogsByRootAuthority","issuer_dlsu_admin_com_1710000000000"]}'
```

Expected Output

```
[{"type":"root_subordinate_approval","orgName":"La Salle Computer Society",...}, ...]
```

2.25 Get Root Sub Rejection Logs by Root Authority

Get rejected-only subordinate logs for a specific root authority.

Template Command (with placeholders)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetRootSubordinateRejectionLogsByRootAuthority","ROOT_AUTHORITY_ID"]}'
```

Actual Command (with example values)

```
peer chaincode query \
 -C chaincert-channel \
 -n chaincert \
 -c '{"Args":["GetRootSubordinateRejectionLogsByRootAuthority","issuer_dlsu_admin_com_1710000000000"]}'
```

Expected Output

```
[{"type":"root_subordinate_rejection",...}, ...]
```

2.26 Get Admin Password Change Logs

Get admin password change logs for a specific admin email.

Template Command (with placeholders)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAdminPasswordChangeLogs","ADMIN_EMAIL"]}'
```

Actual Command (with example values)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAdminPasswordChangeLogs","admin@email.com"]}'
```

Expected Output

```
[{"adminEmail":"admin@email.com","changeType":"password_change",
"timestamp":"2026-03-20T15:00:00Z","txId":"abc123..."}, ...]
```

3. CREATE Operations (Write / Invoke)

3.1 Create Credential Template

Create a new credential template (blueprint for issuing credentials).

Template Command (with placeholders)

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c
'{"Args":["CreateCredentialTemplate","TEMPLATE_ID","ISSUER_ID","CREDENTIAL_TYPE","CREDENTIAL_NAME
","DESCRIPTION","METADATA_JSON","SKILLS_JSON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c
'{"Args":["CreateCredentialTemplate","cred_1710000000000_abc123def","issuer_dlsu_admin_com_1710000000000","Aca
demic","Bachelor of Science in IT","4-year degree program","{}","[\"Programming\",\"Database Management\"]"]}'
```

Expected Output

```
{"id":"cred_1710000000000_abc123def","issuerID":"issuer_dlsu_admin_com_1710000000000",
"credentialType":"Academic","credentialName":"Bachelor of Science in IT",
"status":"active","isTemplate":true,"issuedDate":"2026-03-01T10:00:00Z"}
```

3.2 Issue Credential

Issue a credential instance to a specific holder based on an existing template.

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["IssueCredential","TEMPLATE_ID","HOLDER_ID","EXPIRY_DATE"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["IssueCredential","cred_1710000000000_abc123def","abc123def456ghi789","2027-03-01T00:00:00Z"]}'
```

Expected Output

```
{"id":"cred_1710000000000_abc123def_abc123def456ghi789_1710000100000",
"templateId":"cred_1710000000000_abc123def","holderID":"abc123def456ghi789",
"status":"issued","expiryDate":"2027-03-01T00:00:00Z"}
```

3.3 Revoke Credential

Revoke a credential (sets status to revoked) and store a rejection reason.

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["RevokeCredential","CREDENTIAL_ID","REJECTION_REASON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["RevokeCredential","cred_1710000000000_abc123def_abc123def456ghi789_1710000100000","Academic integrity violation"]}'
```

Expected Output

```
{"id":"cred_...","status":"revoked","revokedAt":"2026-03-20T12:00:00Z",
"rejectionReason":"Academic integrity violation"}
```

3.4 Expire Credential

Mark a credential as expired on-chain.

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
```

```
-C chaincert-channel \
-n chaincert \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "${ORG1_PEER_TLS}" \
--peerAddresses localhost:12051 \
--tlsRootCertFiles "${DLSU_PEER_TLS}" \
-c '{"Args":["ExpireCredential","CREDENTIAL_ID"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["ExpireCredential","cred_1710000000000_abc123def_abc123def456ghi789_1710000100000"]}'
```

Expected Output

```
{"id":"cred_...","status":"expired","expiredAt":"2026-03-23T00:00:00Z"}
```

3.5 Initialize Ledger

Initialize the chaincode (no seed data).

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["InitLedger"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["InitLedger"]}'
```

Expected Output

```
ChainCert chaincode initialized
```

3.6 Create Verification Log

Create a verification audit log entry for a credential verification event.

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c
'{"Args":["LogVerification","CREDENTIAL_ID","VERIFIER_NAME","VERIFIER_EMAIL","VERIFICATION_RESULT_JSON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
```

```
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["LogVerification","cred_1710000000000_abc123def","HR
Department","hr@company.com","{\"valid\":true,\"reason\":\"ok\"}"]}'
```

Expected Output

```
{"logId":"vlog_abc123...","credentialId":"cred_...","verifierName":"HR Department",
"result":{"valid":true,"reason":"ok"},"timestamp":"2026-03-20T09:00:00Z"}
```

3.7 Log Issuer Status Change

Log issuer enable/disable/status change.

Template Command (with placeholders)

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["LogIssuerStatusChange","ISSUER_EMAIL","PREVIOUS_STATUS","NEW_STATUS","REASON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
```

```
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["LogIssuerStatusChange","dlsu@email.com","approved","disabled","Compliance review"]}'
```

Expected Output

```
{"type":"issuer_status_log","email":"dlsu@email.com",
"previousStatus":"approved","newStatus":"disabled","reason":"Compliance review"}
```

3.8 Log User Status Change

Log user enable/disable/status change.

Template Command (with placeholders)

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["LogUserStatusChange","USER_EMAIL","PREVIOUS_STATUS","NEW_STATUS","REASON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["LogUserStatusChange","student@email.com","active","disabled","Account violation"]}'
```

Expected Output

```
{"type":"user_status_log","email":"student@email.com",
"previousStatus":"active","newStatus":"disabled"}
```

3.9 Log Issuer Approval

Log issuer approval decision.

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["LogIssuerApproval","ISSUER_ID","ISSUER_EMAIL","ORG_NAME","REASON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["LogIssuerApproval","issuer_dlsu_admin_com_1710000000000","dlsu@email.com","De La Salle
University","Complete documentation"]}'
```

Expected Output

```
{"type":"issuer_approval","issuerId":"issuer_dlsu_admin_com_1710000000000",
"issuerEmail":"dlsu@email.com","orgName":"De La Salle University"}
```

3.10 Log Issuer Rejection

Log issuer rejection decision.

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["LogIssuerRejection","ISSUER_ID","ISSUER_EMAIL","ORG_NAME","REASON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["LogIssuerRejection","issuer_fake_org_com_1710000000000","fake@org.com","Fake Org","Incomplete documentation"]}'
```

Expected Output

```
{"type":"issuer_rejection","issuerEmail":"fake@org.com",
"orgName":"Fake Org","reason":"Incomplete documentation"}
```

3.11 Log Root Subordinate Approval

Log subordinate approval by root authority.

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
```

```
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c
'{"Args":["LogRootSubordinateApproval","ISSUER_ID","ISSUER_EMAIL","ORG_NAME","ROOT_AUTHORITY_ID",
"ROOT_AUTHORITY_NAME","REASON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c
'{"Args":["LogRootSubordinateApproval","issuer_innocencio_1710000000000","innocencio@dlsu.edu.ph","INNOCENCI
O","issuer_dlsu_admin_com_1710000000000","De La Salle University","Verified department"]}'
```

Expected Output

```
{"type":"root_subordinate_approval","orgName":"INNOCENCIO",
"rootAuthorityName":"De La Salle University","reason":"Verified department"}
```

3.12 Log Root Subordinate Rejection

Log subordinate rejection by root authority.

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
```

```
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c
'{"Args":["LogRootSubordinateRejection","ISSUER_ID","ISSUER_EMAIL","ORG_NAME","ROOT_AUTHORITY_ID"
,"ROOT_AUTHORITY_NAME","REASON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
 --tlsRootCertFiles "${DLSU_PEER_TLS}" \
 -c '{"Args":["LogRootSubordinateRejection","issuer_fake_dept_1710000000000","fakedept@dlsu.edu.ph","Fake
Department","issuer_dlsu_admin_com_1710000000000","De La Salle University","Not a recognized department"]}'
```

Expected Output

```
{"type":"root_subordinate_rejection","orgName":"Fake Department",
"reason":"Not a recognized department"}
```

3.13 Log Admin Password Change

Log admin password change.

Template Command (with placeholders)

```
peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile "${ORDERER_CA}" \
 -C chaincert-channel \
 -n chaincert \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles "${ORG1_PEER_TLS}" \
 --peerAddresses localhost:12051 \
```

```
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["LogAdminChangePassword","ADMIN_EMAIL","CHANGE_TYPE","REASON"]}'
```

Actual Command (with example values)

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["LogAdminChangePassword","admin@email.com","password_change","Routine security update"]}'
```

Expected Output

```
{"type":"admin_password_change","adminEmail":"admin@email.com",
"changeType":"password_change","timestamp":"2026-03-20T15:00:00Z"}
```

Appendix B — Backend File Structure

This section provides an overview of the backend system structure.

B.1 Backend Structure

```
chaincert-api/
├── sdk-server-fixed.js
├── couchDB_service.js
├── .env
├── enrollChaincertAdmin.js
├── setup_Wallet_DLSU.js
├── setup_Wallet_DLSU_Subordinate_INNOCENCIO.js
├── setup_Wallet_DLSU_Subordinate_LASALLECOMPUTERSOCIETY.js
├── controllers/
│   ├── authController.js
│   ├── accountController.js
│   ├── issuerOnboardingController.js
│   ├── issuerCredentialController.js
│   ├── adminIssuerController.js
│   ├── adminAuditController.js
│   ├── adminDashboardController.js
│   ├── rootAuthorityController.js
│   ├── verifierController.js
│   └── walletController.js
├── routes/
│   ├── authRoutes.js
│   ├── adminRoutes.js
│   ├── issuerRoutes.js
│   ├── credentialRoutes.js
│   ├── walletRoutes.js
│   ├── verificationRoutes.js
│   └── systemRoutes.js
├── config/
│   └── issuerRequirements.js
└── wallet/
    ├── chaincert-admin.id
    ├── delasalleuniversity-admin.id
    ├── delasalleuniversity-subordinate-innocencio.id
    └── delasalleuniversity-subordinate-lasallecomputersociety.id
```

B.2 Chaincode Structure

```
chaincert-chaincode/
├── index.js
└── package.json
```

Appendix C — Chaincode Redeployment Instructions

When chaincode (index.js) is modified, the following steps must be executed to deploy the updated version to the Fabric network.

Prerequisites

Stop the backend API server (Ctrl+C in the API terminal) before redeploying. Run the relevant copy commands (see Appendix A of the Installation Manual) for any modified files.

Step 1: Set Fabric environment variables

Directory: ~/fabric-samples/fabric-samples/test-network

```
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=${PWD}/../config/
```

Step 2: Deploy updated chaincode to base network (Org1 + Org2)

```
./network.sh deployCC \
 -c chaincert-channel \
 -ccn chaincert \
 -ccl javascript \
 -ccp ~/chaincert-chaincode \
 -ccv 2.0 \
 -ccs 2
```

Note: Increment the version (-ccv) and sequence (-ccs) with each redeployment. If the previous deployment was version 2.0 sequence 2, use version 3.0 sequence 3, and so on.

Step 3: Copy the DLSU-inclusive configtx

This step is required because deployCC resets configtx.yaml to the default (Org1 + Org2 only). The DLSU-inclusive version must be restored.

```
cp "<PROJECT_ROOT>/backend/node_creation/dlsu/configtx-WITHDLSU.yaml" \
 ~/fabric-samples/fabric-samples/test-network/configtx/configtx.yaml
```

Step 4: Install and approve chaincode on DLSU peer

Set the DLSU peer context:

```
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="DLSUMSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.exa
mple.com/tls/ca.crt
```

```
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/dlsu.example.com/users/Admin@dlsu.exam
ple.com/msp
export CORE_PEER_ADDRESS=localhost:12051
```

Package and install the chaincode on the DLSU peer:

```
peer lifecycle chaincode install chaincert.tar.gz
```

Query the installed chaincode to get the package ID:

```
peer lifecycle chaincode queryinstalled
```

Approve the chaincode for DLSU:

```
peer lifecycle chaincode approveformyorg \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --channelID chaincert-channel \
  --name chaincert \
  --version 2.0 \
  --package-id <PACKAGE_ID_FROM_QUERY> \
  --sequence 2 \
  --tls \
  --cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.co
m-cert.pem"
```

Step 5: Restart the backend API server

```
cd ~/chaincert-api
node sdk-server-fixed.js
```

Design Rationale

The base deployCC script only handles Org1 and Org2. DLSU (and any additional organizations added via node creation) requires separate chaincode installation and approval steps. The configtx-WITHDLSU.yaml must be restored after every deployCC because the script overwrites it with the default two-org configuration.